

SIF-NLP-Precursor Project

Round 1 Evaluation – Backend Architecture & Development Summary

1. My Role

My responsibility is backend architecture and deployment. I am developing the backend layer that connects the client, NLP module, database, and future admin dashboard. The backend receives safety reports, validates data, stores results, provides APIs, and preserves user/HSE feedback.

2. Architecture Implemented

Current working flow: Client → HTTP/JSON → FastAPI → SQLite database → FastAPI response → Client

Target integrated flow: Client → FastAPI → File processing/OCR → NLP model → NLP prediction → SPS → Database → Client/Admin

3. Database Layer – Completed

Component	Purpose
SQLite (data/sif.db)	Development database
SQLAlchemy	ORM and database interaction
incidents	Stores original safety reports
predictions	Stores NLP prediction results and SPS
prediction_feedback	Stores user/HSE review and corrections
clusters	Prepared for future clustering
incident_clusters	Links incidents to clusters

4. Data Validation – Completed

Created Pydantic schemas for ReportInput, NLPResult, SPSBreakdown, feedback, dashboard responses, similar reports, taxonomy, model information, and report history. NLPResult validation was tested successfully. An invalid energy_level value such as **extreme** was correctly rejected; only moderate, high, and very_high are accepted.

5. SPS Calculation – Completed

Implemented SPS calculation as a separate service. The service validates the input and calculates normalized SPS using $\text{raw_total} / \text{max_possible}$. Example: $14 / 20 = 0.70$. Validation protects against invalid maximum values, negative totals, and totals above the maximum.

6. FastAPI APIs – Implemented and Tested

Method	Endpoint	Purpose	Status
GET	/health	Backend health check	Completed
POST	/api/v1/analyze	Receive/store safety report	Completed
GET	/api/v1/analyze	Report history	Completed
GET	/api/v1/{case_id}	Individual report details	Completed
POST	/api/v1/{case_id}/feedback	User/HSE feedback	Completed
POST	/api/v2/dashboard	Dashboard statistics	Completed
GET	/api/v1/similar-reports/{case_id}/similar	Similar report interface	Placeholder

Method	Endpoint	Purpose	Status
GET	/api/v1/categories/taxonomy	System categories	Completed
GET	/api/v1/model/model-info	Model information	Completed

7. Client–Backend Communication – Completed

The client uses HTTP through httpx to communicate with FastAPI. A safety report can be sent as JSON, and the server generates the case ID. A successful test returned CASE-0000002 for the report “A worker slipped near a chemical storage area.” The same record was retrieved through the report-history API.

8. Feedback System – Completed

A user/HSE reviewer can mark a prediction as correct or incorrect. Feedback is stored separately in prediction_feedback so the original NLP prediction is preserved. Corrections and comments can be stored for incorrect predictions.

9. Dashboard – Completed

The dashboard API aggregates database information. A successful test returned 2 reports, 1 prediction, 1 high-risk report, 0 medium-risk reports, 0 low-risk reports, and 3 correct feedback records. The feedback count reflects multiple test submissions.

10. Similar Reports – Interface Prepared

The similar-report API and service structure are implemented, but the actual NLP/embedding similarity algorithm has not yet been integrated. The current service returns a placeholder similarity score of 0.0. The interface is ready for the future similarity model.

11. What Is Still Pending

- Actual NLP model integration with /api/v1/analyze
- NLP-generated report title and precursor prediction
- PDF text extraction/OCR
- DOCX processing
- ZIP/batch report processing
- Actual embedding-based similar-report algorithm
- Clustering algorithm and severity/action analysis
- Admin filtering by SPS and categories
- Production deployment and final integration

12. One-Minute Explanation for Faculty

My role is backend architecture and deployment. I developed the backend using FastAPI, SQLAlchemy, and SQLite. The client sends safety reports as JSON through HTTP, FastAPI validates and stores them, and the server generates the case ID. I designed separate database tables for incidents, NLP predictions, feedback, and future clustering. I created Pydantic schemas that define the contract between the NLP module and backend, including prediction fields, confidence, evidence, and SPS. I implemented SPS normalization, report history, individual report retrieval, HSE feedback, dashboard aggregation, taxonomy, model-information, and a similarity-report interface. I have tested the client-to-FastAPI-to-database flow successfully. The remaining work is integrating the actual NLP model, OCR and batch processing, real similarity detection, clustering, admin filtering, and final deployment.

Current assessment: The backend infrastructure, database layer, validation layer, API layer, and client-to-server communication are implemented and tested. AI/document-processing components are pending integration.